Strix: How It Works and How It Helps with Automation

What Strix is

Strix is an open-source tool that runs autonomous AI "pentesters." You give it a target (a code folder, a GitHub repo, a live URL, or an API spec) and an LLM key, and it tests the target the way a human pentester would. It then reports only vulnerabilities it has actually exploited with a working proof of concept (PoC).

How it works

1. Root agent plus subagents (the "Graph of Agents")

- A root agent doesn't touch the target itself. It reads the scope, decides how to split the work, and spawns specialized subagents for recon, vulnerability hunting (injection, access control, auth, business logic), exploit validation, and reporting.
- Subagents run in parallel and can be spawned later as findings come in, for example a new agent to chain two bugs together.
- strix/core/agents.py tracks each agent's status, parent, message mailbox, and budget.

2. Shared threat model

- Before the hunters start, the root agent writes one shared "who is the attacker and what are they attacking" document. For black-box targets, it writes this after recon.
- Subagents correct it as they learn more, so five agents don't each build their own picture of the target.

3. Sandbox with real tools

- Everything runs in a Kali-based Docker container with the usual toolchain: Nmap, ffuf, Nuclei, SQLMap, subfinder, httpx, Katana, jwt_tool, TruffleHog, and others.
- Agents also get a Caido HTTP proxy for intercepting and replaying requests, a Playwright browser for XSS, CSRF, and auth flows, a terminal, and a Python runtime for writing custom exploit scripts.

4. Skills (knowledge packs)

- The strix/skills/ folder holds Markdown playbooks for vulnerability classes (IDOR, JWT, SSRF, SQLi, deserialization), frameworks (Django, FastAPI, Next.js), protocols (OAuth, GraphQL), and tools. Agents load them only when relevant, which keeps prompts small and the testing focused.

5. Validation and reporting

- An agent files a finding only after proving it with a PoC. Duplicates are merged, and severity and coverage are tracked.
- Output goes to strix_runs/<run>/ as a Markdown report, per-vulnerability files, JSON, and SARIF (the format GitHub's code scanning understands).

6. Scan modes

- Quick takes minutes, standard 30 to 60 minutes, and deep 1 to 4 hours. White-box runs also do source-aware static analysis (Opengrep, AST search, secrets, dependency checks) to decide what to attack dynamically first.

How it helps with automation

- **CI/CD gating:** strix -n --scan-mode quick --fail-on high runs headless, scopes itself to changed files on pull requests, and returns exit code 2 if it finds something serious. That can block a merge automatically.
- **Fewer false positives:** Traditional scanners flag "possible" issues. Strix only reports what its agents exploited, so there's less triage work.
- **Parallel coverage:** Many agents test different attack surfaces at once, so a test that takes a human days can finish in hours.
- **Repeatable pipeline:** It goes from recon to exploit to report to patch suggestion with no manual steps. SARIF output plugs into existing dashboards.
- **Bug bounty and recon automation:** It handles the repetitive enumeration and PoC-building so a human can focus on creative attack ideas.
- **Agent-friendly:** It can be driven from Claude Code, Cursor, and similar tools through its skills/ folder and MCP integration.

Connecting this to your projects

- Strix is a good **target for VulnForge**. Running it against your own deliberately vulnerable app gives you a benchmark: which of your planted bugs it finds, which it misses, and why. That makes a strong write-up. Only point it at systems you own or are authorized to test.
- For your own agent project, the parts worth studying are the root-agent coordination skill (skills/coordination/root_agent.md), the agent coordinator (core/agents.py), and the shared threat model and coverage tools.